

Linguaggi Formali e Compilatori

(Formal Languages and Compilers)

prof. S. Crespi Reghizzi, prof. Angelo Morzenti
(prof. Luca Breveglieri)

Prova scritta - 2 febbraio 2012 - Parte I: Teoria

CON SOLUZIONI - A SCOPO DIDATTICO LE SOLUZIONI SONO MOLTO ESTESE E COMMENTATE VARIAMENTE - NON SI RICHIEDE CHE IL CANDIDATO SVOLGA IL COMPITO IN MODO ALTRETTANTO AMPIO, BENSÌ CHE RISPONDA IN MODO APPROPRIATO E A SUO GIUDIZIO RAGIONEVOLE

NOME:

MATRICOLA:

FIRMA:

ISTRUZIONI - LEGGERE CON ATTENZIONE:

- L'esame si compone di due parti:
 - I (80%) Teoria:
 1. espressioni regolari e automi finiti
 2. grammatiche libere e automi a pila
 3. analisi sintattica e parsificatori
 4. traduzione sintattica e analisi semantica
 - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve sostenere con successo entrambe le parti (I e II), in un solo appello oppure in appelli diversi, ma entro un anno.
- Per superare la parte I (teoria) occorre dimostrare di possedere conoscenza sufficiente di tutte le quattro sezioni (1-4), rispondendo alle domande obbligatorie.
- È permesso consultare libri e appunti personali.
- Per scrivere si utilizzi lo spazio libero e se occorre anche il tergo del foglio; è vietato allegare nuovi fogli o sostituirne di esistenti.
- Tempo: Parte I (teoria): 2h.30m - Parte II (esercitazioni): 45m

1 Espressioni regolari e automi finiti 20%

1. È data la grammatica lineare a destra G seguente (assioma S), con alfabeto terminale $\{ a, b \}$:

$$G \left\{ \begin{array}{l} S \rightarrow b P \mid b Q \mid \varepsilon \\ P \rightarrow (a \mid b) Q \mid a S \\ Q \rightarrow a S \mid \varepsilon \end{array} \right.$$

Si risponda alle domande seguenti:

- (a) Si ricavi un'espressione regolare R che genera il linguaggio (regolare) $L(G)$.
 - (b) (facoltativa) Se l'espressione regolare R contiene più operatori stella, la si semplifichi per ottenerne una R' equivalente che contiene un solo operatore stella.
-

Soluzione

(a) Vanno risolte le tre equazioni insiemistiche lineari seguenti:

$$(1) \quad S = bP \cup bQ \cup \varepsilon$$

$$(2) \quad P = (a \cup b)Q \cup aS$$

$$(3) \quad Q = aS \cup \varepsilon$$

Si sostituisce l'equazione (3) nella (2), poi distribuendo e raccogliendo si ottiene:

$$\begin{aligned} P &= (a \cup b)(aS \cup \varepsilon) \cup aS \\ &= aaS \cup baS \cup a \cup b \cup aS \\ &= (a \cup b \cup \varepsilon)aS \cup a \cup b \\ &= \beta aS \cup \alpha \end{aligned}$$

dove per brevità si pone $\alpha = a \cup b$ e $\beta = a \cup b \cup \varepsilon$, e si noti che valgono le relazioni $\alpha \cup \varepsilon = \beta$ e $\beta \cup \varepsilon = \beta$ (servono sotto).

Sostituendo le espressioni di P e Q nell'equazione (1) si ottiene:

$$S = b(\beta aS \cup \alpha) \cup b(aS \cup \varepsilon) \cup \varepsilon \quad (1)$$

$$= b(\beta aS \cup \alpha \cup aS \cup \varepsilon) \cup \varepsilon \quad (2)$$

$$= b(\beta aS \cup \beta \cup aS) \cup \varepsilon \quad (3)$$

$$= b(\beta aS \cup \beta) \cup \varepsilon \quad (4)$$

$$= b\beta aS \cup b\beta \cup \varepsilon \quad (5)$$

$$= (b\beta a)S \cup (b\beta \cup \varepsilon) \quad (6)$$

poiché si ha $\alpha \cup \varepsilon = \beta$ per passare dalla riga (2) alla (3) e si ha $\beta aS \cup aS = (\beta \cup \varepsilon)aS = \beta aS$ per passare dalla riga (3) alla (4); i passaggi rimanenti sono semplici raccoglimenti o distribuzioni.

Infine applicando l'identità di Arden alla riga (6) si ottiene la soluzione di S :

$$S = (b\beta a)^*(b\beta \cup \varepsilon)$$

Sostituendo la definizione di β si ha l'espressione regolare R del linguaggio $L(G)$:

$$\begin{aligned} R &= (b(a \mid b \mid \varepsilon)a)^*(b(a \mid b \mid \varepsilon) \mid \varepsilon) \\ &= (baa \mid bba \mid ba)^*(ba \mid bb \mid b \mid \varepsilon) \\ &= (baa \mid bba \mid ba)^*[ba \mid bb \mid b] \end{aligned}$$

In alternativa si potrebbe disegnare l'automa equivalente alla grammatica e applicare il metodo di eliminazione dei nodi (BMC).

(b) Qui l'operatore stella nell'espressione regolare R trovata è già uno solo e dunque non occorre semplificare oltre. Per R ci possono essere altre forme equivalenti.

2. Si consideri l'espressione regolare R seguente, con alfabeto $\{ a, s, t \}$:

$$R = (a t)^+ \left(s (a t)^+ \right)^*$$

Si risponda alle domande seguenti:

- (a) Tramite il metodo di Berry-Sethi (BS), si costruisca un automa deterministico A che riconosce il linguaggio regolare $L(R)$.
 - (b) Si verifichi se l'automa deterministico A è minimo e in caso negativo si ricavi l'automa minimo A' equivalente ad A .
-

Soluzione

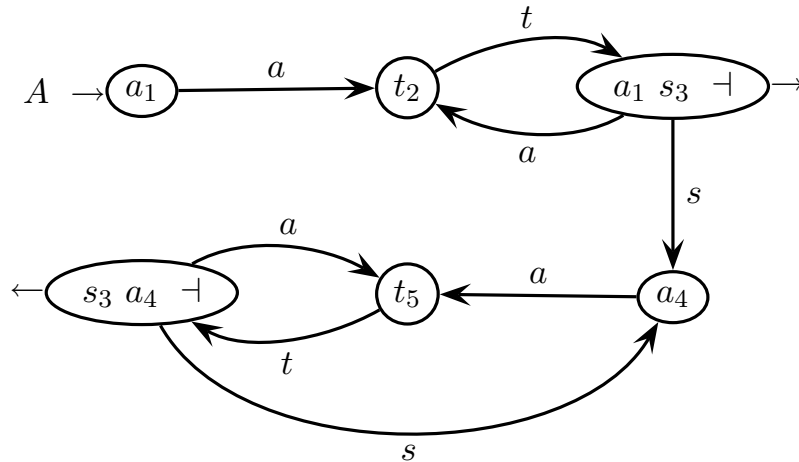
- (a) Ecco il metodo di Berry-Sethi, del tutto standard. Prima si scrive l'espressione regolare $R_{\#}$ con generatori numerati e terminatore:

$$R_{\#} = (a_1 t_2)^+ (s_3 (a_4 t_5)^+)^* \dashv$$

Poi si fa l'analisi degli inizi e dei seguiti:

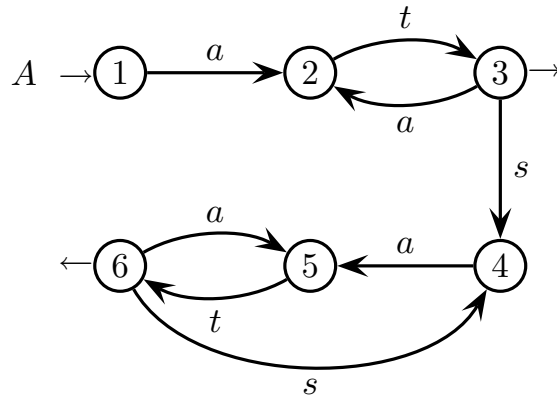
inizi	a_1
generatore	seguiti
a_1	t_2
t_2	$a_1 s_3 \dashv$
s_3	a_4
a_4	t_5
t_5	$s_3 a_4 \dashv$

E infine si disegna il grafo stato-transizione dell'automa deterministico A equivalente all'espressione regolare R :



L'automa A così trovato ha sei stati di cui due finali, ed è deterministico ma non necessariamente minimo.

- (b) L'automa deterministico A non è in forma minima, come si constata rapidamente costruendo la tabella triangolare dei vincoli d'indistinguibilità. Prima si rinominano gli stati del grafo di A , per semplicità:



Poi si compila e risolve la tabella triangolare dei vincoli d'indistinguibilità:

2	×				
3	×	×			
4	2 5	×	×		
5	×	3 6	×	×	
6	×	×	2 5	×	×
	1	2	3	4	5

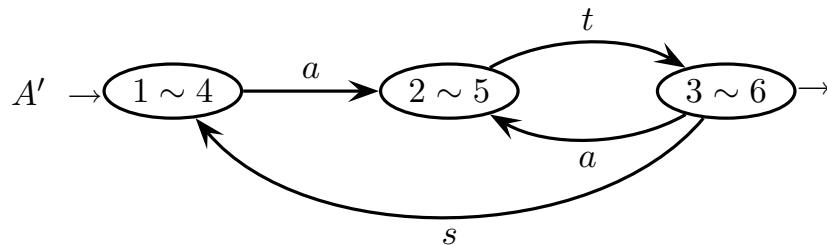
 $\Rightarrow$

2	×				
3	×	×			
4	~	×	×		
5	×	~	×	×	
6	×	×	~	×	×
	1	2	3	4	5

vincoli d'indistinguibilità
tabella risolta

Da questa tabella si vede che i vincoli 3 6 e 2 5 sono soddisfatti circolarmente e dunque valgono le equivalenze $2 \sim 5$ e $3 \sim 6$ (per l'eq. $3 \sim 6$ ci sarebbe anche il vincolo 4 4 che però è soddisfatto banalmente), e che pertanto vale anche l'equivalenza $1 \sim 4$ vincolata all'equivalenza $2 \sim 5$.

Infine unendo gli stati equivalenti si ha il grafo dell'automa deterministico minimo A' equivalente ad A :



L'automa deterministico minimo A' ha tre stati di cui uno finale, ed è unico.

2 Grammatiche libere e automi a pila 20%

1. Sono date le due grammatiche G_1 e G_2 seguenti (assiomi S), entrambe con alfabeto terminale $\{ a, b, c \}$:

$$G_1 \left\{ \begin{array}{l} S \rightarrow a S X S \mid \varepsilon \\ X \rightarrow b \mid c \end{array} \right. \quad G_2 \left\{ \begin{array}{l} S \rightarrow X c S \mid X c \\ X \rightarrow a \mid b \end{array} \right.$$

Si risponda alle domande seguenti:

- (a) Si costruisca una grammatica G che genera il linguaggio concatenamento $L(G)$:

$$L(G) = L(G_1) \cdot L(G_2)$$

- (b) Si verifichi se la grammatica G così costruita è ambigua, in caso positivo mostrando una stringa ambigua di G e giustificando l'ambiguità di tale stringa.
- (c) (facoltativa) Se la grammatica G è ambigua, si costruisca una grammatica G' non ambigua equivalente a G .
-

Soluzione

- (a) Ecco la grammatica G (assioma S), disgiungendo gli alfabeti nonterminali:

$$G \left\{ \begin{array}{l} S \rightarrow A B \\ \hline A \rightarrow a A X A \mid \varepsilon \\ X \rightarrow b \mid c \\ \hline B \rightarrow Y c B \mid Y c \\ Y \rightarrow a \mid b \end{array} \right.$$

- (b) La grammatica G_1 ha la nota forma di Dyck non ambigua, e la grammatica G_2 è lineare a destra e pure non ambigua. È facile vedere che $L(G_1)$ è il linguaggio di Dyck (con ε) con parentesi aperta a e chiusa indifferentemente b o c , e che $L(G_2)$ è una lista non vuota di coppie di parentesi non annidate, ossia $()() \dots ()$, con parentesi aperta indifferentemente a o b e chiusa c . Il linguaggio $L(G)$ è senza ε . Pertanto le due stringhe più brevi di $L(G)$ sono $\varepsilon a c = a c$ e $\varepsilon b c = b c$, e non sono ambigue. Ma la stringa $a c a c$ è ambigua poiché si ha:

$$\underbrace{a c}_{G_1} \underbrace{a c}_{G_2} \quad \underbrace{\varepsilon}_{G_1} \underbrace{a c}_{G_2} \underbrace{a c}_{G_2}$$

Si tratta di ambiguità di concatenamento. Le stringhe $(a c)^n$ (con $n \geq 2$) sono tutte ambigue (non sono le sole) e hanno grado di ambiguità crescente con n .

- (c) Bisogna risolvere l'ambiguità di concatenamento. Un modo consiste nel forzare la grammatica G_2 a generare subito almeno una coppia $b c$ poiché questa non è generabile da parte di G_1 , oltre a eventuali coppie $a c$ e $b c$ che la seguono. Ma occorre anche notare che la coppia $b c$ è facoltativa nel linguaggio $L(G)$ e ricordare che questo è senza ε . Ecco dunque questa versione della grammatica G' (assioma S), dove per chiarezza si eliminano i nonterminali X e Y :

$$G' \left\{ \begin{array}{l} \text{-- regole assiomatiche} \\ S \rightarrow A B \mid A a A c \\ \hline \text{-- linguaggio di Dyck con } \varepsilon \text{ con } (= a e) = b, c \\ A \rightarrow a A b A \mid a A c A \mid \varepsilon \\ \hline \text{-- lista non vuota con } (= a, b e) = c \text{ e inizio } b c \\ B \rightarrow b c C \\ C \rightarrow a c C \mid b c C \mid \varepsilon \end{array} \right.$$

Si vede subito che G' non genera ε e che genera sia $a c$ sia $b c$ non ambigualmente. Si noti inoltre che in G' la stringa $a c a c$ è generata non ambigualmente dalla nuova regola assiomatica $S \rightarrow A a A c$ che è la variante ricorsiva a sinistra della regola standard di Dyck non ambigua. Tale regola genera non ambigualmente ogni altra stringa di $L(G)$ senza coppia $b c$ e che termina con una struttura parentetica di tipo $a \dots c$, e che dunque non può derivare dalla regola $S \rightarrow A B$.

2. Il linguaggio da modellare è una piccola parte dell'inglese con qualche semplificazione, illustrata dagli esempi seguenti:

NounPlural *ADJective*
 books are cheap clever and nice

a $\overbrace{\text{reader}}$ $\overbrace{\text{buys}}$ *a book which is short nice and cheap*

young $\overbrace{\text{travelers}}^{\text{NounPlural}}$ $\overbrace{\text{board}}^{\text{TransitiveVerbPlural}}$ a red $\overbrace{\text{bus}}^{\text{NounSingular}}$ which is not full

dogs eat tasty and big bones which are fresh

a dog which is mad bytes boys which *IntransitiveVerbPlural*
run

I modelli di frase sono questi:

soggetto verbo intransitivo

soggetto verbo transitivo oggetto

soggetto *is o are* aggettivi

L'alfabeto terminale comprende le parole sottolineate e le classi lessicali indicate, cioè:

ADJ *NS* *NP* *IVS* *IVP* *TVS* *TVP*

Per esempio la classe *TVS* sta per i verbi transitivi in terza persona singolare, come *eats*, *buys*, ecc. Il nome singolare dev'essere preceduto da articolo *a*. Il nome (singolare o plurale) può essere preceduto da uno o più aggettivi e può essere seguito da una sola frase relativa *which*. Le liste di aggettivi sono terminate da *and*.

Le concordanze da rispettare sono queste:

- (a) soggetto singolare/plurale $\iff$ verbo singolare/plurale
- (b) verbo transitivo/intransitivo $\iff$ con (o senza)/senza complemento oggetto
- (c) *is* e *are* sono seguiti da uno o più aggettivi eventualmente negati
- (d) anche nelle frasi relative: soggetto singolare/plurale $\iff$ verbo singolare/plurale

Si scriva una grammatica EBNF non ambigua per questo linguaggio.

Soluzione

Ecco la grammatica EBNF richiesta (assioma PHRASE):

$\langle \text{PHRASE} \rangle$	$\rightarrow$	$\langle \text{NPHR_S} \rangle \langle \text{KERN_S} \rangle \mid \langle \text{NPHR_P} \rangle \langle \text{KERN_P} \rangle$
$\langle \text{NPHR_S} \rangle$	$\rightarrow$	$\text{a} \left[\langle \text{ADJ_LST} \rangle \right] \langle \text{NS} \rangle \left[\text{which} \langle \text{KERN_S} \rangle \right]$
$\langle \text{NPHR_P} \rangle$	$\rightarrow$	$\left[\langle \text{ADJ_LST} \rangle \right] \langle \text{NP} \rangle \left[\text{which} \langle \text{KERN_P} \rangle \right]$
$\langle \text{KERN_S} \rangle$	$\rightarrow$	$\langle \text{IVS} \rangle \mid \langle \text{TVS} \rangle \left[\langle \text{NPHR} \rangle \right] \mid \text{is} \langle \text{nADJ_LST} \rangle$
$\langle \text{KERN_P} \rangle$	$\rightarrow$	$\langle \text{IVP} \rangle \mid \langle \text{TVP} \rangle \left[\langle \text{NPHR} \rangle \right] \mid \text{are} \langle \text{nADJ_LST} \rangle$
$\langle \text{ADJ_LST} \rangle$	$\rightarrow$	$\langle \text{ADJ} \rangle \mid \langle \text{ADJ} \rangle^+ \text{and} \langle \text{ADJ} \rangle$
$\langle \text{nADJ_LST} \rangle$	$\rightarrow$	$\langle \text{nADJ} \rangle \mid \langle \text{nADJ} \rangle^+ \text{and} \langle \text{nADJ} \rangle$
$\langle \text{NPHR} \rangle$	$\rightarrow$	$\langle \text{NPHR_S} \rangle \mid \langle \text{NPHR_P} \rangle$
$\langle \text{nADJ} \rangle$	$\rightarrow$	$\left[\text{not} \right] \langle \text{ADJ} \rangle$
$\langle \text{ADJ} \rangle$	$\rightarrow$	set of adjectives
$\langle \text{NS} \rangle$	$\rightarrow$	set of singular nouns
$\langle \text{NP} \rangle$	$\rightarrow$	set of plural nouns
$\langle \text{IVS} \rangle$	$\rightarrow$	set of singular intransitive verbs
$\langle \text{IVP} \rangle$	$\rightarrow$	set of plural intransitive verbs
$\langle \text{TVS} \rangle$	$\rightarrow$	set of singular transitive verbs
$\langle \text{TVP} \rangle$	$\rightarrow$	set of plural transitive verbs

Qui si segue il modello cosiddetto “a nucleo” della frase: soggetto + nucleo; il soggetto è una frase nominale e il nucleo è una frase predicativa ossia un predicato (verbo) seguito dai suoi oggetti in ordine prefissato e stabilito dal predicato stesso. La classe sintattica **NPHR** genera le frasi nominali (noun phrase) che consistono di nome singolare con articolo o plurale opzionalmente espanso con aggettivi e frase relativa. La classe sintattica **KERN** genera i nuclei di frase (kernel) che consistono di predicato singolare o plurale seguito dagli oggetti obbligatori od opzionali come previsto dal predicato, i quali a loro volta sono frasi nominali o aggettivi opzionalmente negati se il predicato è la copula. Entrambe le classi sono divise per numero onde concordare soggetto e nucleo. Aggettivi nomi e predicati sono infine espansi nei rispettivi dizionari, qui modellati come insiemi di lessemi da elencare esplicitamente. Si veda qualunque grammatica moderna per maggiori dettagli sul modello a nucleo della frase. Naturalmente sono possibili anche altri approcci modellistici alla frase. La grammatica non è ambigua poiché è costituita da componenti non ambigue.

3 Analisi sintattica e parsificatori 20%

È data la grammatica estesa (EBNF) G seguente (assioma S), con alfabeto terminale $\{ a, b, c, d \}$:

$$G \left\{ \begin{array}{l} S \rightarrow (X d)^+ \\ X \rightarrow a(Y b \mid \varepsilon) \mid \varepsilon \\ Y \rightarrow c(X \mid c) \mid a \end{array} \right.$$

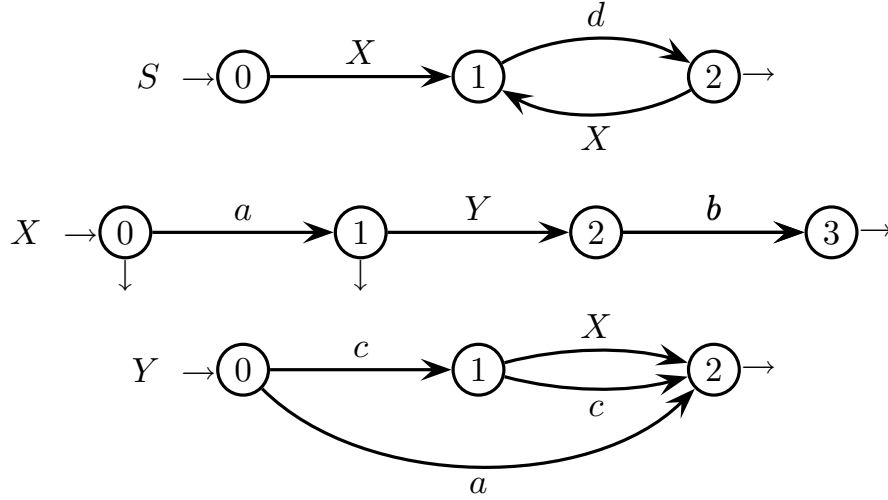
Si risponda alle domande seguenti mediante la metodologia di analisi sintattica estesa:

1. Si tracci la rete ricorsiva di automi a stati finiti che rappresentano le regole di G .
2. Si tracci il grafo pilota di G e si mostri che la condizione ELR è soddisfatta, motivando la risposta.
3. Si mostri che anche la condizione ELL è soddisfatta da G , motivando la risposta.
4. (facoltativa) Sulla rete ricorsiva di automi finiti di G si traccino gli archi di chiamata (call arcs) con i rispettivi insiemi guida (guide sets), e negli stati finali (stati di riduzione) si indichino gli insiemi di prospezione (look-ahead sets).

Nota: chi preferisse rispondere alle domande secondo la metodologia classica di analisi sintattica (condizioni LR e LL), metta la grammatica in forma non estesa (BNF) modificando così la regola assiomatica: $S \rightarrow X d S \mid X d$.

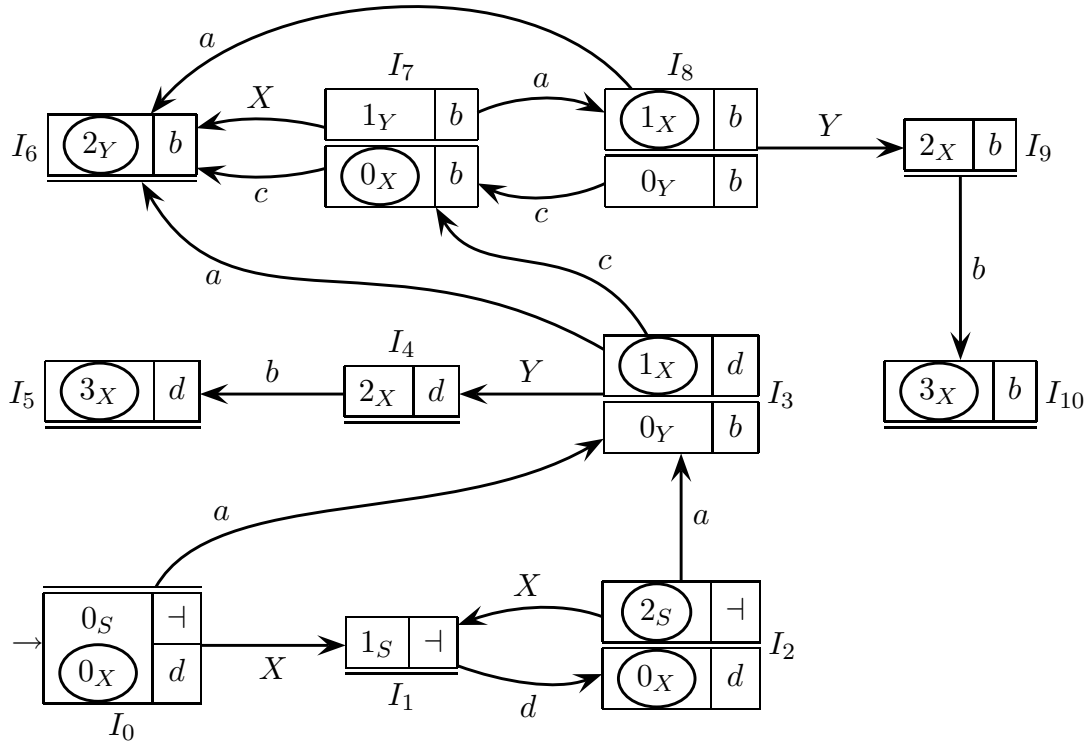
Soluzione

1. Ecco la rete ricorsiva di automi a stati finiti (sull'alfabeto unione di terminali e nonterminali) o macchine che rappresentano la grammatica estesa G :



Queste macchine sono deterministiche e in forma minima, e gli stati iniziali non hanno archi entranti. Per brevità qui i nomi degli stati non hanno pedici.

2. Ecco il grafo pilota esteso della grammatica G , ricavato dalla rappresentazione di G come rete ricorsiva di macchine (si ricorda che $X \equiv 0_X$ e $Y \equiv 0_Y$):

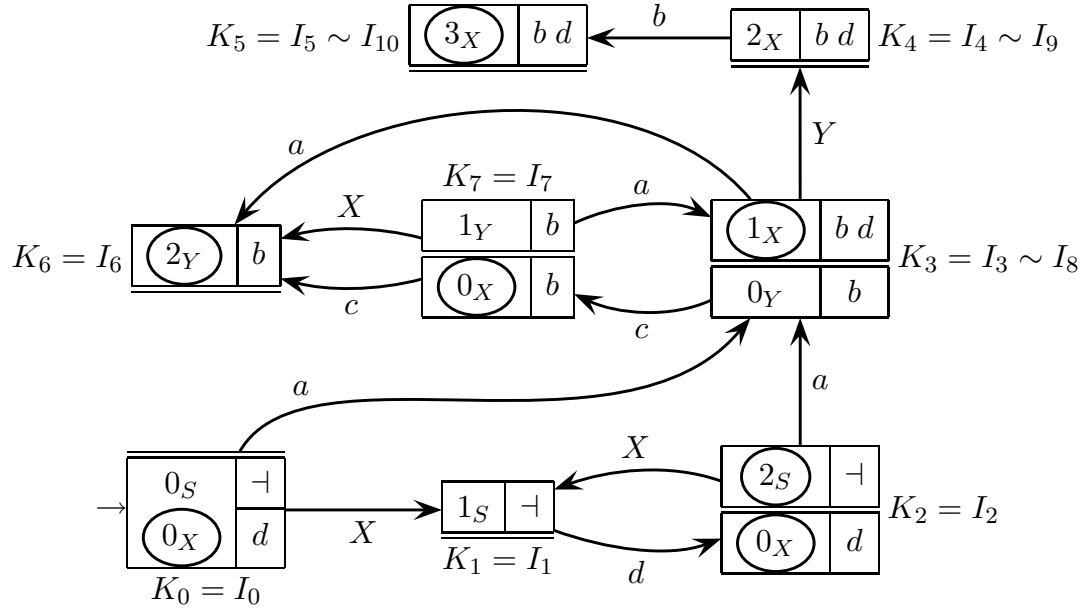


Ecco l'analisi dei potenziali conflitti nel grafo pilota, passando in esame gli m -stati:

- in I_0 la riduzione 0_X ha look-ahead d e non ci sono archi uscenti con d
- in I_2 le riduzioni 2_S e 0_X hanno look-ahead $\neg$ e d che sono disgiunti, e non ci sono archi uscenti con d (e ovviamente neppure con $\neg$)
- in I_3 la riduzione 1_X ha look-ahead d e non ci sono archi uscenti con d
- in I_7 la riduzione 0_X ha look-ahead b e non ci sono archi uscenti con b
- in I_8 la riduzione 1_X ha look-ahead b e non ci sono archi uscenti con b
- in I_5 I_6 e I_{10} ci sono solo riduzioni isolate (m -stati di riduzione pura)
- in I_1 I_4 e I_9 non ci sono riduzioni (m -stati di spostamento puro)

Pertanto il grafo pilota non ha conflitti né di tipo riduzione-spostamento né di tipo riduzione-riduzione. Dunque la grammatica G soddisfa la condizione $ELR(1)$.

Per completezza qui si dà anche il grafo pilota compatto, ottenuto sovrapponendo gli m -stati kernel-equivalenti e unendo i look-ahead rispettivi. Eccolo:



Il grafo pilota compatto ha solo otto stati (qui rinominati K_i con $0 \leq i \leq 7$) e alcuni look-ahead appaiono uniti. Beninteso pure il pilota compatto soddisfa la condizione ELR poiché deriva da un pilota esteso che ha già tale proprietà.

3. Riesaminando il grafo pilota della grammatica estesa G , si vede quanto segue:

- il grafo pilota soddisfa la condizione $ELR(1)$, come discusso prima
- le basi di tutti gli m -stati tranne I_0 contengono esattamente una candidata e la base dello m -stato iniziale I_0 è vuota (condizione di Beatty)
- la rete ricorsiva di automi grammaticali non presenta derivazioni ricorsive a sinistra: basta osservare che al massimo l'assioma S va completato con il non-terminale X , ma la catena di completamento finisce su X

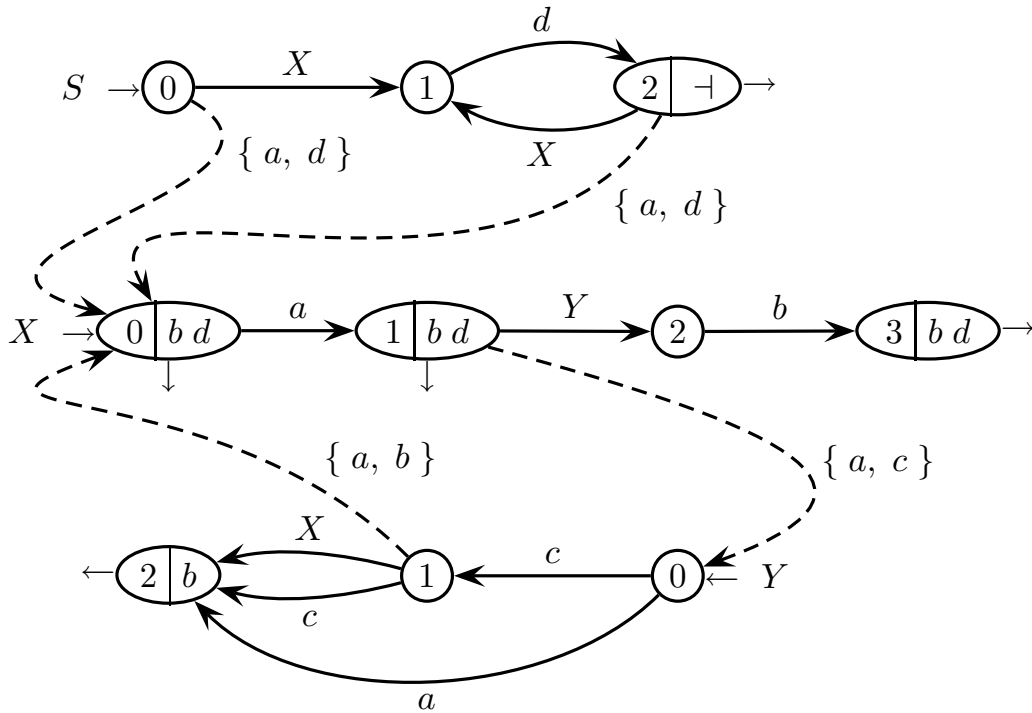
Pertanto la grammatica G soddisfa le tre condizioni che prese insieme sono necessarie e sufficienti per avere la proprietà ELL , e dunque è di tipo $ELL(1)$.

Si noti che anche il grafo pilota compatto soddisfa la proprietà ELL ed è di tipo $ELL(1)$. Infatti non ha conflitti ELR , come osservato prima, soddisfa la condizione di Beatty poiché sovrapporre stati kernel-equivalenti con basi unitarie le lascia unitarie, e naturalmente non ha ricorsioni sinistre poiché la grammatica estesa è la stessa.

4. È noto che se il grafo pilota soddisfa la condizione ELL , lo si può semplificare nella rete ricorsiva degli automi grammaticali collegati tramite archi di chiamata (call arcs). Ogni stato della rete risulta dall'unione delle candidate omonime nel grafo pilota. In primo luogo interessano gli insiemi di prospezione (look-ahead sets) degli stati finali ossia delle candidate di riduzione nell'analisi ELR . Qui si ha quanto segue:

- la candidata 2_S in I_2 ha look-ahead $\{ \neg \}$
- le due candidate 0_X in I_2 e I_7 hanno look-ahead unificato $\{ b, d \}$
- le due candidate 1_X in I_3 e I_8 hanno look-ahead unificato $\{ b, d \}$
- le due candidate 3_X in I_5 e I_{10} hanno look-ahead unificato $\{ b, d \}$
- la candidata 2_Y in I_6 ha look-ahead $\{ b \}$

Poi si possono etichettare gli stati finali della rete ricorsiva con i look-ahead elencati e tracciare gli archi di chiamata con i rispettivi insiemi guida (guide sets). Così si ottiene il grafo pilota semplificato dell'analizzatore sintattico a discesa ricorsiva (recursive descent parser), forma di grafo pilota topologicamente identica alla rete ricorsiva di automi grammaticali e resa possibile proprio dalla condizione ELL :



Ecco come calcolare gli insiemi guida (guide sets) sugli archi di chiamata (call arcs):

- su arco di chiamata $0_S \dashrightarrow 0_X$: unione del carattere terminale a sull'arco di spostamento che esce da 0_X e del carattere terminale d sull'arco di spostamento che esce da 1_S , successore dello spostamento nonterminale $0_S \xrightarrow{X} 1_S$, poiché X è nullabile
- su arco di chiamata $2_S \dashrightarrow 0_X$: unione del carattere terminale a sull'arco di spostamento che esce da 0_X e del carattere terminale d sull'arco di spostamento che esce da 1_S , successore dello spostamento nonterminale $2_S \xrightarrow{X} 1_S$, poiché X è nullabile
- su arco di chiamata $1_Y \dashrightarrow 0_X$: unione del carattere terminale a sull'arco di spostamento che esce da 0_X e del carattere terminale b dell'insieme di prospezione (look-ahead) dello stato finale 2_Y , successore dello spostamento nonterminale $1_Y \xrightarrow{X} 2_Y$, poiché X è nullabile
- su arco di chiamata $1_X \dashrightarrow 0_Y$: unione dei caratteri terminali a e c sugli archi di spostamento che escono da 0_Y

Qui non succede di avere due o più archi di chiamata concatenati poiché non ci sono catene di completamento che abbiano lunghezza maggiore di uno.

La grammatica è *ELL* e dunque l'analizzatore a discesa ricorsiva è deterministico. Per esempio si noti che l'insieme di prospezione (look-ahead) $\{ b, d \}$ nello stato finale 1_X (che si può considerare come un insieme guida applicato alla freccetta di uscita che marca 1_X come stato finale) è disgiunto dall'insieme guida (guide set) $\{ a, c \}$ sull'arco di chiamata a Y che esce da 1_X , altrimenti l'analizzatore non saprebbe se terminare l'analisi di X oppure sospenderla passando a quella di Y . E similmente il guide set $\{ a, b \}$ sull'arco di chiamata a X che esce dallo stato 1_Y (non finale) è disgiunto dal carattere terminale c (che si può considerare come un insieme guida singoletto $\{ c \}$) sull'arco di spostamento che esce da 1_Y , altrimenti l'analizzatore non saprebbe se sospendere l'analisi di Y passando a quella di X oppure continuarla spostandosi nello stato 2_Y . Complessivamente il grafo dell'analizzatore a discesa ricorsiva non ha conflitti gli insiemi guida sugli archi di spostamento terminale (continui) e sugli archi di chiamata (tratteggiati), e con gli insiemi di prospezione (look-ahead) negli stati finali (o se si vuole insiemi guida di uscita sulle freccette di stato finale).

Qui si riporta lo svolgimento con la metodologia classica *LR* e *LL* di analisi sintattica.

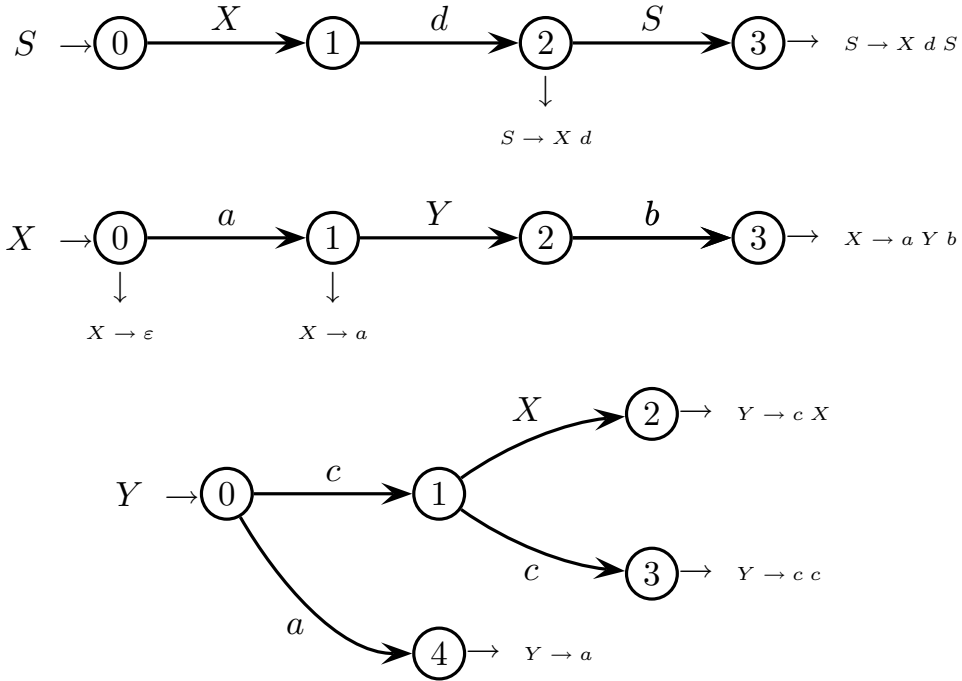
1. Per l'analisi *LR* classica, ecco la grammatica G' in forma non estesa (BNF):

$$S \rightarrow (X d)^+ \quad X \rightarrow a(Y b \mid \varepsilon) \mid \varepsilon \quad Y \rightarrow c(X \mid c) \mid a$$

$$G' \left\{ \begin{array}{lll} S \rightarrow X d & X \rightarrow \varepsilon & Y \rightarrow a \\ S \rightarrow X d S & X \rightarrow a & Y \rightarrow c c \\ & X \rightarrow a Y b & Y \rightarrow c X \end{array} \right.$$

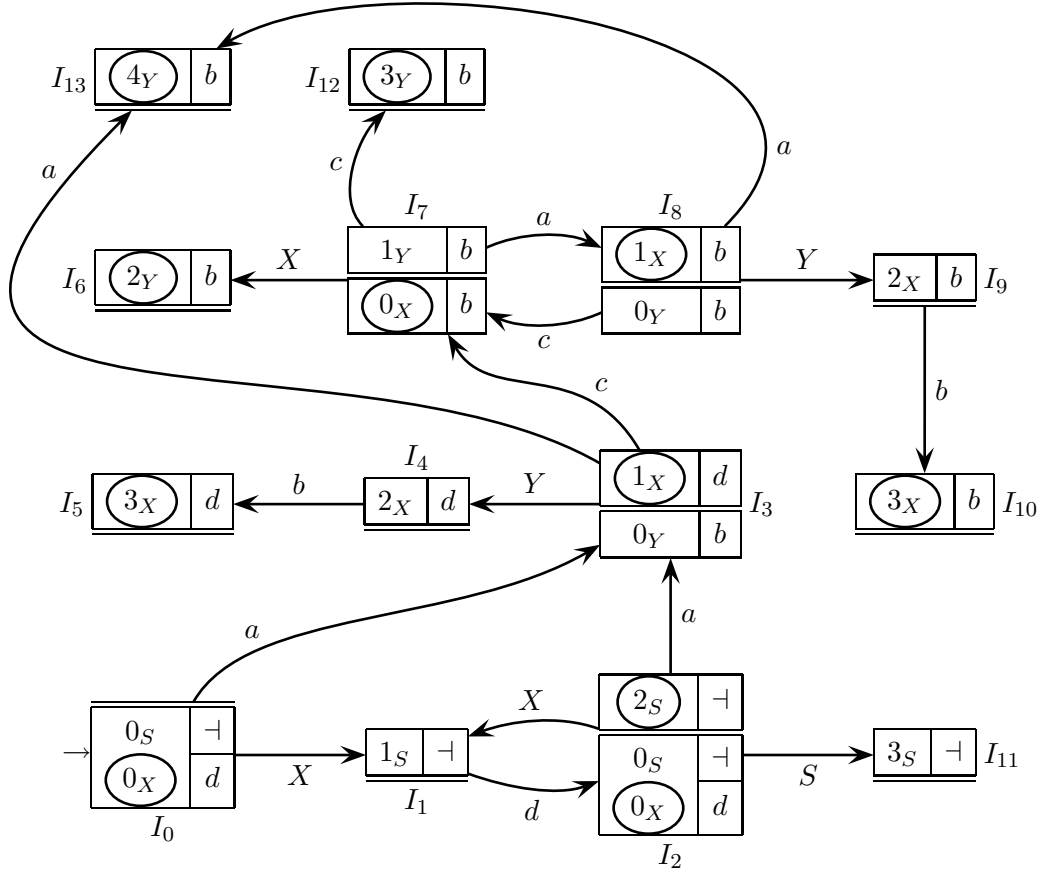
Le regole alternative di G' sono raggruppate sotto le regole estese di G da cui derivano. Le regola estesa assiomatica è qui ridotta a ricorsione destra, come suggerisce il testo.

Ed ecco le macchine deterministiche ad albero senza cicli che rappresentano la grammatica G' in forma non estesa (BNF) (senza pedici agli stati):



Gli stati finali di ciascuna macchina identificano i percorsi che li raggiungono (e gli automi non sono minimi) o equivalentemente le regole alternative della grammatica. L'automa di X è invariato rispetto alla forma estesa G .

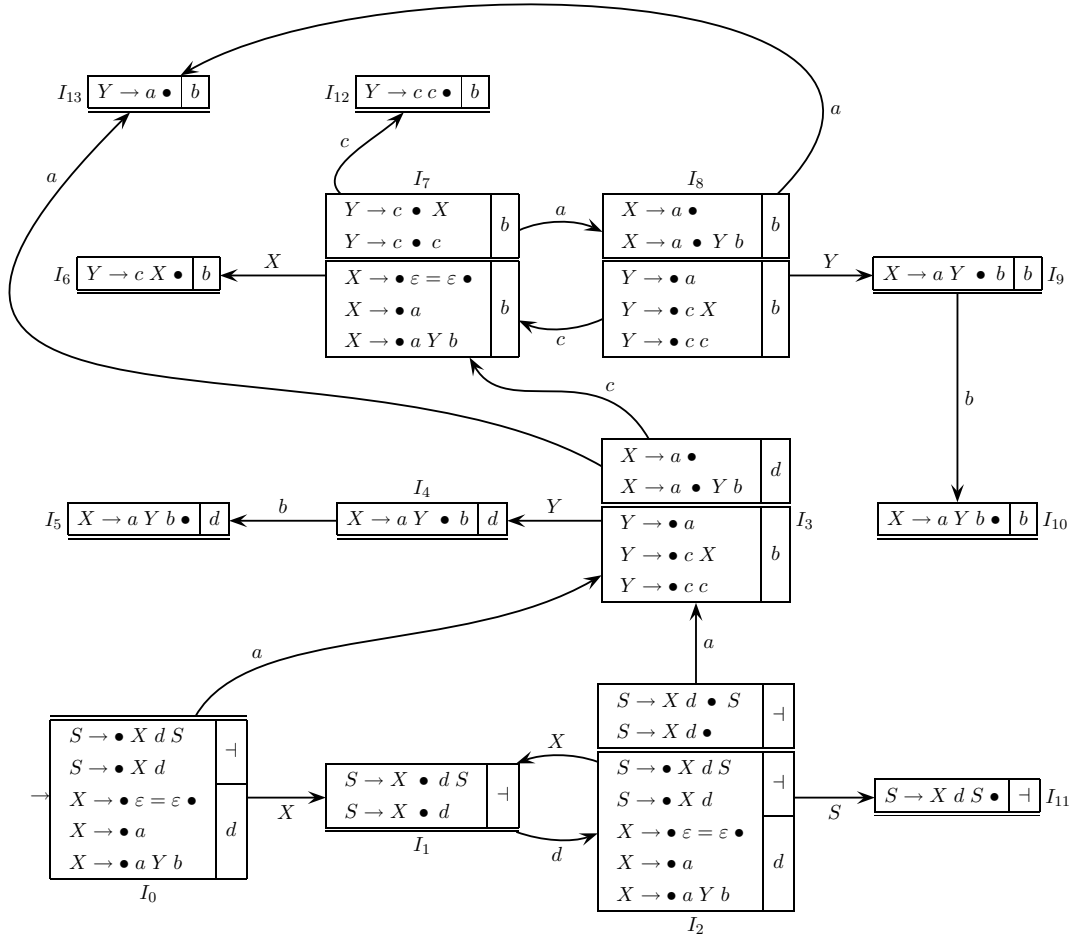
2. Ecco il grafo pilota classico, ricavato con riferimento alle macchine ad albero senza cicli della grammatica G' in forma non estesa (BNF):



Gli stati finali sono cerchiati. Il grafo pilota classico ha quattordici m -stati ed è molto simile a quello esteso che ne ha solo undici. La grammatica G' in forma non estesa (BNF) risulta facilmente essere di tipo $LR(1)$ poiché la condizione LR è verificata: non ci sono conflitti né di tipo riduzione-spostamento né di tipo riduzione-riduzione.

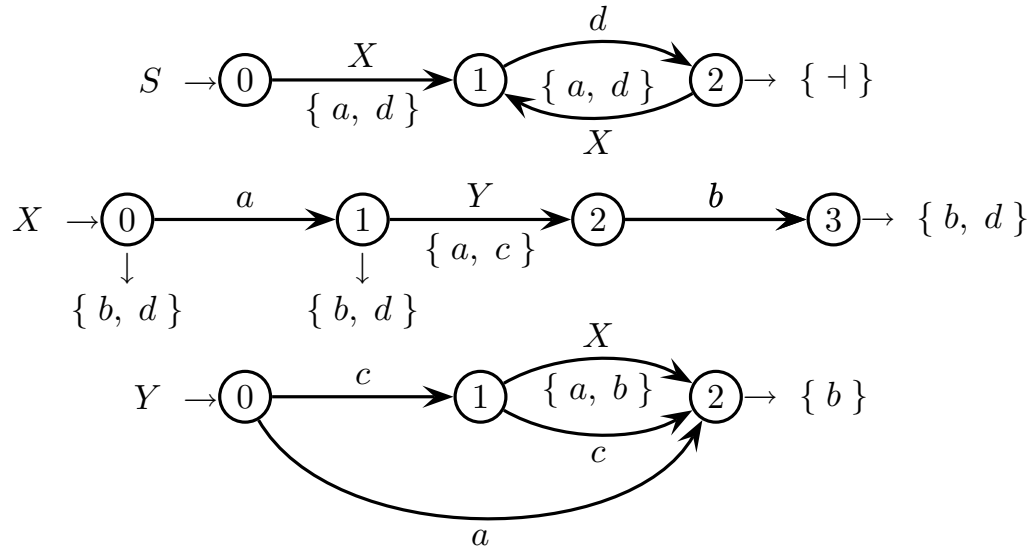
Beninteso si potrebbe dare il grafo pilota compatto (sovrapponendo gli m -stati kernel-equivalenti) anche per la grammatica non estesa G' (BNF). Qui si avrebbero le kernel-equivalenze $I_3 \sim I_8$, $I_4 \sim I_9$ e $I_5 \sim I_{10}$. Il risultato è meno compatto che nel caso esteso poiché qui si ha maggiore dispersione di stati (in quanto le macchine non sono minime). Inoltre nell'analisi sintattica classica il pilota compatto non è di utilità poiché l'analisi LL è indipendente dall'analisi LR e non un caso particolare.

Negli m -stati, invece degli stati si possono mettere le regole marcate corrispondenti. Per esempio in I_0 la candidata $(0_S \mid \neg)$ si espande nelle due regole marcate $S \rightarrow \bullet X d S$ e $S \rightarrow \bullet X d$ (iniziali), entrambe con look-ahead $\neg$; in I_1 la candidata $(1_S \mid \neg)$ si espande nelle due regole marcate $S \rightarrow X \bullet d S$ e $S \rightarrow X \bullet d$ (spostamento terminale), entrambe con look-ahead $\neg$; in I_3 la candidata $(1_X \mid d)$ si espande nelle due regole marcate $X \rightarrow a \bullet$ (riduzione) e $X \rightarrow a \bullet Y b$ (spostamento nonterminale), entrambe con look-ahead d ; e così via. Ecco il risultato, un po' opprimente:



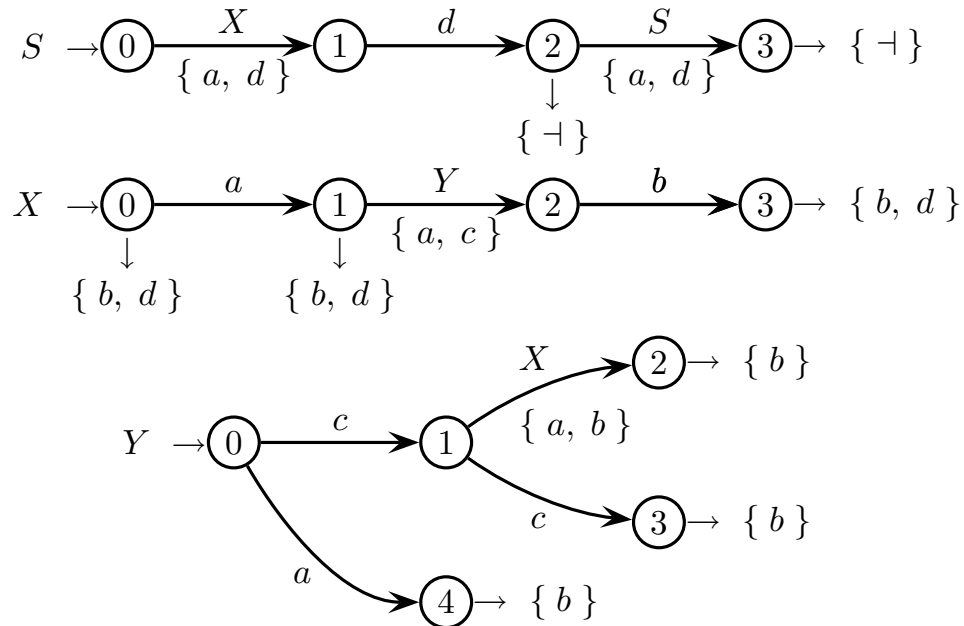
Beninteso questa rappresentazione con regole ha le stesse proprietà di quella con stati.

3. Si può condurre l'analisi classica *LL* direttamente sulle macchine con cicli della grammatica estesa G (EBNF). Eccola:



Si vede facilmente che la grammatica estesa (EBNF) è di tipo *LL*(1) poichè la condizione classica *LL* è soddisfatta: non ci sono conflitti tra gli insiemi guida sugli archi nonterminali e i caratteri sugli archi terminali uscenti da uno stesso stato.

In alternativa si può anche condurre l'analisi classica *LL* sulla versione non estesa G' (BNF) della grammatica, rappresentata tramite macchine ad albero senza cicli. Eccola:



Gli insiemi guida sono identici e la conclusione pure: la grammatica non estesa G' è di tipo *LL*(1). Comunque ciò è complicato dato che le macchine della grammatica non estesa (BNF) hanno più stati di quelli della grammatica estesa (EBNF).

4 Traduzione e analisi semantica 20%

1. Per il linguaggio sorgente regolare $L = a^+ b^*$ si consideri la traduzione τ seguente:

$$\begin{aligned}\tau(a^{2m} b^n) &= c^m & m \geq 1 \quad n \geq 0 \\ \tau(a^{2m+1} b^n) &= c^{2m+1} d^n & m \geq 0 \quad n \geq 0\end{aligned}$$

Esempi:

$$\begin{aligned}a a a b b &\xrightarrow{\tau} c c & m = 2 \quad n = 2 \\ a a a b b &\xrightarrow{\tau} c c c d d & m = 1 \quad n = 2\end{aligned}$$

Si risponda alle domande seguenti:

- (a) Si progetti uno schema di traduzione sintattico che calcola la traduzione τ e si disegnino gli alberi sintattici che corrispondono alla coppia $a a b \xrightarrow{\tau} c$.
 - (b) (facoltativa) Si progetti un traduttore a stati finiti (automa con due nastri o IO-automa) non necessariamente deterministico oppure un'espressione regolare (razionale) di trasduzione per calcolare la traduzione τ .
-

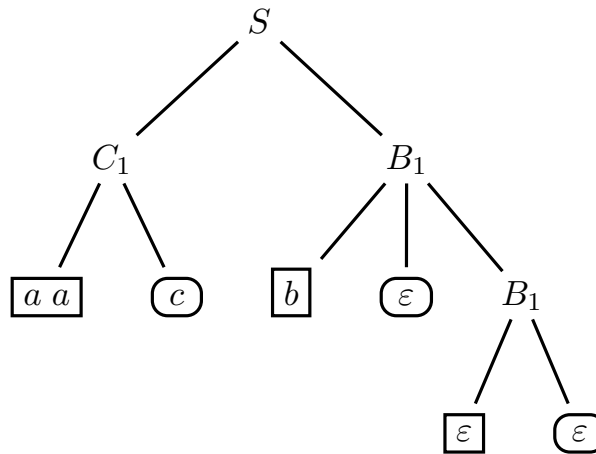
Soluzione

(a) Ecco uno schema di trasduzione che calcola la traduzione τ (assioma S):

G_1	G_2	G_T
$S \rightarrow C_1 B_1$	$S \rightarrow C_1 B_1$	$S \rightarrow C_1 B_1$
$S \rightarrow C_2 B_2$	$S \rightarrow C_2 B_2$	$S \rightarrow C_2 B_2$
$C_1 \rightarrow a a C_1$	$C_1 \rightarrow c C_1$	$C_1 \rightarrow \frac{a a}{c} C_1$
$C_1 \rightarrow a a$	$C_1 \rightarrow c$	$C_1 \rightarrow \frac{a a}{c}$
$C_2 \rightarrow a a C_2$	$C_2 \rightarrow c c C_2$	$C_2 \rightarrow \frac{a a}{c c} C_2$
$C_2 \rightarrow a$	$C_2 \rightarrow c$	$C_2 \rightarrow \frac{a}{c}$
$B_1 \rightarrow b B_1$	$B_1 \rightarrow B_1$	$B_1 \rightarrow \frac{b}{\varepsilon} B_1$
$B_1 \rightarrow \varepsilon$	$B_1 \rightarrow \varepsilon$	$B_1 \rightarrow \frac{\varepsilon}{\varepsilon}$
$B_2 \rightarrow b B_2$	$B_2 \rightarrow d B_2$	$B_2 \rightarrow \frac{b}{d} B_2$
$B_2 \rightarrow \varepsilon$	$B_2 \rightarrow \varepsilon$	$B_2 \rightarrow \frac{\varepsilon}{\varepsilon}$

Questo ovvio schema sintattico (o la grammatica di traduzione corrispondente) traduce indipendentemente e in parallelo le lettere a in c e le lettere b in d .

Il lettore può tracciare da sé gli alberi sintattici, ma ecco qua nella forma unificata con frontiere sorgente (riquadri) e destinazione (cartigli) sullo stesso albero:

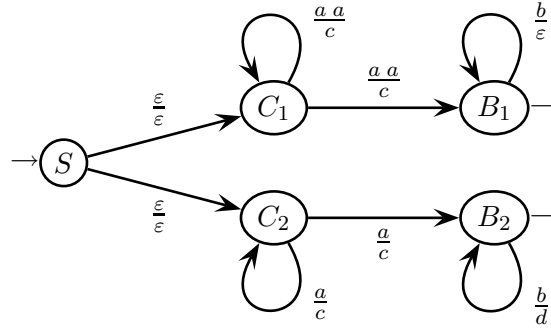


Naturalmente si può rappresentare come due alberi sintattici separati e strutturalmente identici tranne per avere frontiere sugli alfabeti sorgente e destinazione, rispettivamente.

- (b) La grammatica di traduzione è quasi completamente nella forma lineare a destra, tranne per le due regole assiomatiche. Con una semplice modifica si ottiene lo schema di trasduzione che ha grammatica di traduzione lineare a destra:

G'_1	G'_2	G'_T
$S \rightarrow C_1$	$S \rightarrow C_1$	$S \rightarrow C_1$
$S \rightarrow C_2$	$S \rightarrow C_2$	$S \rightarrow C_2$
$C_1 \rightarrow a a C_1$	$C_1 \rightarrow c C_1$	$C_1 \rightarrow \frac{a a}{c} C_1$
$C_1 \rightarrow a a B_1$	$C_1 \rightarrow c B_1$	$C_1 \rightarrow \frac{a a}{c} B_1$
$C_2 \rightarrow a a C_2$	$C_2 \rightarrow c c C_2$	$C_2 \rightarrow \frac{a a}{c c} C_2$
$C_2 \rightarrow a B_2$	$C_2 \rightarrow c B_2$	$C_2 \rightarrow \frac{a}{c} B_2$
$B_1 \rightarrow b B_1$	$B_1 \rightarrow B_1$	$B_1 \rightarrow \frac{b}{\varepsilon} B_1$
$B_1 \rightarrow \varepsilon$	$B_1 \rightarrow \varepsilon$	$B_1 \rightarrow \frac{\varepsilon}{\varepsilon}$
$B_2 \rightarrow b B_2$	$B_2 \rightarrow d B_2$	$B_2 \rightarrow \frac{b}{d} B_2$
$B_2 \rightarrow \varepsilon$	$B_2 \rightarrow \varepsilon$	$B_2 \rightarrow \frac{\varepsilon}{\varepsilon}$

La modifica consiste nel mettere la derivazione di B_1 in serie a quella di C_1 , invece che in parallelo; similmente per B_2 e C_2 . Da questo schema si traccia immediatamente il grafo dell'automa trasduttore. Esso ha i simboli nonterminali come stati e le transizioni corrispondono alle regole della grammatica G'_T . Gli stati finali sono B_1 e B_2 , e corrispondono alle regole nulle. Ecco il grafo:



Il traduttore a stati finiti così ottenuto non è deterministico ed è arduo pensare ne esista uno tale: per decidere se emettere uno o due caratteri c per ogni carattere a in ingresso esso dovrebbe prima valutare la parità della sequenza di caratteri a in ingresso, la quale ha lunghezza arbitraria, e poi emettere il giusto numero di caratteri c ; ma ciò va oltre la portata di un modello di automa con memoria limitata e nastro d'ingresso unidirezionale.

Tuttavia non sarebbe difficile ricavare un automa deterministico pur di usare la pila: prima legge e impila i caratteri a valutandone la parità; poi li spila emettendo i caratteri c secondo la parità; infine legge i caratteri b (senza impilarli) ed emette o no altrettanti caratteri d secondo la parità, terminando a stato finale. Quanto all'espressione razionale di traduzione R_τ che realizza τ , eccone una:

$$R_\tau = \left(\frac{a a}{c} \right)^+ \left(\frac{b}{\varepsilon} \right)^* \mid \frac{a}{c} \left(\frac{a a}{c c} \right)^* \left(\frac{b}{d} \right)^*$$

L'espressione R_τ è molto intuitiva e si giustifica da sé. Inoltre non è ambigua.

2. Tramite una grammatica con attributi si modelli la traduzione in codice macchina dello statement di controllo **break** che tronca il ciclo **while** più *interno* contenente **break** e salta al primo statement a valle del ciclo ossia a quello che segue **end**. Per tradurre **break** si usa l'istruzione macchina di salto incondizionato a etichetta.

È data la sintassi G seguente, dove figurano espansi solo gli statement rilevanti per l'esercizio. La sintassi G *non* assicura che ogni statement **break** sia incluso in un ciclo **while**: in caso di **break** fuori ciclo la sua traduzione è la stringa **error**.

$$\begin{aligned}
 S &\rightarrow \langle \text{LIST} \rangle \quad // \text{ regola assiomatica} \\
 \langle \text{LIST} \rangle &\rightarrow \langle \text{STM} \rangle ; \langle \text{LIST} \rangle \\
 \langle \text{LIST} \rangle &\rightarrow \langle \text{STM} \rangle \\
 \langle \text{STM} \rangle &\rightarrow \text{while } C \text{ do } \langle \text{LIST} \rangle \text{ end } \langle \text{STM} \rangle \\
 \langle \text{STM} \rangle &\rightarrow \text{break} \\
 \langle \text{STM} \rangle &\rightarrow \dots \quad // \text{ altri stm. - non interessano} \\
 C &\rightarrow \dots \quad // \text{ cond. ciclo - non interessa}
 \end{aligned}$$

Allo scopo si usino gli attributi seguenti qui descritti informalmente (a parole):

τ	codice macchina risultante dalla traduzione - si possono concatenare più traduzioni parziali tramite l'operatore “•”
n	numero intero incorporato nell'etichetta che identifica l'istruzione macchina destinazione di salto - contribuisce a rendere univoca ogni etichetta d'istruzione macchina
in_wh	booleano che ha valore logico vero se e solo se ci si trova all'interno di un ciclo while (altrimenti ha valore falso)

A questi se ne possono aggiungere altri eventuali secondo quanto si chiede di fare.

Per tradurre **break** si usi l'istruzione macchina di salto incondizionato seguente:

jump stm_ n

Essa salta all'etichetta **stm_** n che identifica la destinazione (lo stm. a valle del ciclo), dove n è un intero univoco. Per generare l'etichetta si ricorra alla funzione *new_* n che per ogni invocazione restituisce un intero positivo n differente dai precedenti.

Si risponda alle domande seguenti:

- Si definiscano compiutamente gli attributi necessari e si scrivano le funzioni semantiche per calcolare la traduzione (si vedano le tabelle predisposte).
- (facoltativa) Si consideri lo statement **smash** che tronca il ciclo **while** più *esterno* contenente **smash** (e dunque pure quelli interni) e salta allo statement a valle del ciclo più esterno. Con sintesi e precisione si tratteggino informalmente modifiche e aggiunte da apportare alla risposta precedente per tradurre **smash**.

attributi da usare per la grammatica

tipo	nome	(non)terminali	dominio	significato
	τ		stringa	codice macchina risultante dalla traduzione
	n		intero	intero univoco incorporato nell'etichetta <code>stm_n</code> applicata allo statement a valle del corpo di ciclo <code>while</code>
	in_wh		booleano	vero se e solo se lo statement (o la lista di statement) è contenuto in (almeno) un ciclo <code>while</code>

sintassi

calcolo attributi

$S_0 \rightarrow \langle \text{LIST} \rangle_1$

$\langle \text{LIST} \rangle_0 \rightarrow \langle \text{STM} \rangle_1 ; \langle \text{LIST} \rangle_2$

$\langle \text{LIST} \rangle_0 \rightarrow \langle \text{STM} \rangle_1$

$\langle \text{STM} \rangle_0 \rightarrow$ while C do
 $\langle \text{LIST} \rangle_1$
end
 $\langle \text{STM} \rangle_2$

$\langle \text{STM} \rangle_0 \rightarrow \text{break}$

Soluzione

- (a) Ogni statement a valle di corpo di ciclo **while** va etichettato in modo univoco per raggiungerlo da **break** (o meglio va etichettata l'istruzione macchina iniziale della sequenza d'istruzioni che realizzano lo statement - soltanto in casi semplici lo statement è realizzato da una sola istruzione macchina). Poi l'etichetta applicata allo statement a valle del ciclo (o meglio il numero intero univoco n incorporato in essa) va propagata tramite l'attributo n allo statement **break** dove viene usata come destinazione nell'istruzione macchina **jump** di salto incondizionato che realizza **break**. Pertanto l'attributo n è ereditato (destro). Poiché secondo la sintassi G data lo statement **break** potrebbe non essere contenuto in nessun ciclo **while**, si usa un attributo aggiuntivo booleano chiamato in_wh (pure ereditato) per controllare questo aspetto. L'attributo τ raccoglie concatenandole tutte le traduzioni parziali ed è sintetizzato (sinistro). Non servono altri attributi.

Ecco le definizioni complete dei tre attributi da usare, con tipo e nonterminali:

attributi da usare per la grammatica

tipo	nome	(non)terminali	dominio	significato
sin	τ	$S \langle \text{LIST} \rangle \langle \text{STM} \rangle$	stringa	codice macchina risultante dalla traduzione
des	n	$\langle \text{LIST} \rangle \langle \text{STM} \rangle$	intero	intero univoco incorporato nell'etichetta stm_n applicata allo statement a valle del corpo di ciclo while
des	in_wh	$\langle \text{LIST} \rangle \langle \text{STM} \rangle$	booleano	vero se e solo se lo statement (o la lista di statement) è contenuto in (almeno) un ciclo while

Se lo statement **break** non si trova in un ciclo **while**, la traduzione contiene la parola chiave **error**. Per il resto le funzioni semantiche usano i tre attributi τ , n e in_wh definiti prima e sono facili da interpretare. Esse realizzano solo quanto serve per tradurre lo statement **break**, mentre tutti gli altri statement come pure condizione e iterazione del ciclo **while** qui sono trascurati. Eccole:

sintassi	calcolo attributi
$S_0 \rightarrow \langle \text{LIST} \rangle_1$	– eredità (fuori ciclo) $in_wh_1 = \text{false}$ $\boxed{n_1 = -1}$ – sintesi $\tau_0 = \tau_1$
$\langle \text{LIST} \rangle_0 \rightarrow \langle \text{STM} \rangle_1 ; \langle \text{LIST} \rangle_2$	– eredità $in_wh_1 = in_wh_0$ $in_wh_2 = in_wh_0$ $\boxed{n_1 = n_0}$ $\boxed{n_2 = n_0}$ – sintesi $\tau_0 = \tau_1 \bullet \tau_2$
$\langle \text{LIST} \rangle_0 \rightarrow \langle \text{STM} \rangle_1$	– eredità $in_wh_1 = in_wh_0$ $\boxed{n_1 = n_0}$ – sintesi $\tau_0 = \tau_1$
$\langle \text{STM} \rangle_0 \rightarrow \text{while } C \text{ do } \langle \text{LIST} \rangle_1 \text{ end } \langle \text{STM} \rangle_2$	– eredità $in_wh_1 = \text{true}$ $in_wh_2 = in_wh_0$ $\boxed{n_1 = new_n}$ $\boxed{n_2 = n_0}$ – sintesi (applica eti.) $\boxed{\tau_0 = \tau_1 \bullet stm_n_1 \bullet \tau_2}$
$\langle \text{STM} \rangle_0 \rightarrow \text{break}$	– sintesi (salta a stm.) if in_wh_0 then $\boxed{\tau_0 = \text{jump } stm_n_0}$ else (fuori ciclo) $\tau_0 = \text{error}$ endif

I riquadri illustrano come tradurre **break** applicando un’etichetta allo statement a valle di **while** e generando l’istruzione **jump** di salto a tale statement. I cartigli illustrano come calcolare e propagare l’intero n che realizza l’etichetta.

Fuori da ciclo **while** l'attributo in_wh ha valore falso (e l'attributo n ha valore negativo -1) e così segnala un eventuale uso errato dello statement **break** (incorporando **error** nella traduzione). Si può ottimizzare sfruttando direttamente il segno dell'attributo n per stabilire se l'istruzione corrente si trovi dentro un ciclo **while**: così nella funzione semantica associata alla regola **break**, in luogo della condizione in_wh_0 si potrebbe scrivere la condizione $n_0 > 0$ rendendo superfluo l'attributo in_wh . Qui si trascura la traduzione degli statement non rilevanti per l'esercizio e non figurano neppure le regole che li generano.

La grammatica con attributi è aciclica e dunque calcolabile. Prima si scende dalla radice alle foglie dell'albero sintattico propagando gli attributi in_wh e n che vengono ridefiniti univocamente (come costante **true** e tramite la funzione new_n) a valle di ogni ciclo **while** (nella radice ossia fuori da **while** sono inizializzati a **false** e -1). Poi si risale dalle foglie calcolando la traduzione e concatenandone tutti gli elementi fino ad averla completa nella radice. Pertanto si conclude che la grammatica è di tipo con una passata (one-sweep).

- (b) Occorrono due attributi distinti (entrambi ereditati) per l'etichetta di fine ciclo: uno chiamato n^{out} per l'etichetta di ciclo **while** esterno; e uno chiamato n^{in} per quelle dei cicli **while** interni, i quali sono uno o più d'uno secondo la profondità di annidamento dei cicli. L'attributo n^{in} funziona come l'attributo n di prima e il cambio di nome qui serve solo per chiarezza espositiva. L'attributo n^{out} è un po' diverso, come si vede qui sotto. I due attributi hanno lo stesso valore negli statement dentro un solo ciclo **while** (che è ciclo esterno) e si differenziano in quelli dentro due o più cicli **while** annidati. Gli altri attributi (in_wh e τ) funzionano come prima. Qui sotto si riportano solo le funzioni semantiche rilevanti.

sintassi	calcolo attributi
$\langle STM \rangle_0 \rightarrow$ while C do $\langle LIST \rangle_1$ end $\langle STM \rangle_2$	– eredità (solo le funz. rilevanti) if in_wh_0 then – ciclo interno $n_1^{out} = n_0^{out}$ else – ciclo esterno $n_1^{out} = n_1^{in}$ endif $n_2^{out} = n_0^{out}$
$\langle STM \rangle_0 \rightarrow$ smash	– sintesi (salta a stm.) if in_wh_0 then $\tau_0 = \text{jump } stm_n_0^{out}$ else (fuori ciclo) $\tau_0 = \text{error}$ endif

Se il ciclo **while** è interno ad altri cicli l'attributo n^{out} della classe sintattica **LIST** (corpo del ciclo interno) mantiene il valore dato all'etichetta di uscita del ciclo più esterno. Se invece il ciclo è esterno l'attributo assume il valore di n^{in} poiché qui gli statement **smash** e **break** sono equivalenti. Questa grammatica con attributi è anch'essa aciclica e di tipo con una passata (one-sweep).